Performance Analysis Report

Introduction

This report analyzes the performance of the Peer-to-Peer File Distribution System implemented with dual trackers, synchronized via a shared log. The analysis covers system behavior under different workloads, such as varying file sizes and peer availability, and evaluates scalability, fault tolerance, and efficiency of resource usage.

Methodology

To evaluate performance, multiple experiments were conducted:
1. Uploading and downloading small (1MB), medium (50MB), and large (500MB+) files.
2. Running simultaneous downloads from multiple clients.
3. Testing fault tolerance by shutting down one tracker mid-download.
4. Measuring responsiveness when multiple users join groups and upload files concurrently.

Observations

- **File Size Scaling**: Small files downloaded almost instantly, while larger files showed linear increases in completion time. The chunking mechanism (512KB pieces) allowed for parallel downloads, which reduced bottlenecks.

- **Concurrency**: The system efficiently handled multiple simultaneous downloads. Each file's pieces were distributed among available peers using a round-robin peer selection algorithm, ensuring fair utilization of peers.

- **Fault Tolerance**: When one tracker failed, the client automatically reconnected to the second tracker without disrupting ongoing downloads. State consistency was maintained via the synchronized log file.

- **Resource Utilization**: Threads were created per downloading file, with efficient use of `pwrite` for concurrent piece writes. Memory overhead remained minimal since only active pieces were buffered.

Analysis

The round-robin distribution of file pieces among peers balanced network usage and improved throughput compared to random assignment. The shared log design, though simple, ensured strong consistency across trackers without requiring complex consensus protocols.

However, as the number of peers increases significantly, replaying the log could become a

scalability bottleneck. Additionally, thread-per-download may stress resources under very high concurrency and could be optimized with thread pooling.

Conclusion

The implemented system demonstrates strong performance across different workloads, with reliable fault tolerance and efficient parallelization. It is well-suited for moderate-scale file distribution environments. Future improvements could focus on optimizing log replay, introducing peer reputation systems for better scheduling, and adopting thread pools to handle very high concurrency scenarios.